

Maragal_8 - Requirements and Restrictions

Computational, numerical and methodological conditions for reproduction

1. Purpose

This document records the conditions that matter when reproducing the Maragal_8 benchmark. Numerical settings that define the experiment should remain fixed for a strict reproduction. Computing-resource settings must instead be checked against the current University CSF policy.

2. Matrix requirements

Property	Required value
Dataset	SuiteSparse Matrix Collection, NYPA/Maragal_8
Rows	33,212
Columns	75,077
Nonzeros	1,308,415
Numerical rank	15,343
Null-space dimension	17,869
Type	Real, rectangular, sparse, rank deficient

The same Maragal_8 matrix must be used. A dense conversion, a different matrix, or a modified set of entries changes the numerical problem.

3. Right-hand side

The supplied program constructs b as $A @ \text{ones}$. This is a fixed part of the benchmark. It is not an observed right-hand side from the original NYPA application.

4. Solver implementations

Solver	Required implementation
PCG	CG on the regularised normal equations; A transpose A is applied matrix-free and a Jacobi diagonal preconditioner is used.
LSQR	Damped LSQR with $\text{damp} = \sqrt{\lambda}$.
LSMR	Damped LSMR with $\text{damp} = \sqrt{\lambda}$.

Do not replace the PCG implementation with an explicitly formed A transpose A if the intention is to reproduce this experiment. The supplied implementation applies A and A transpose as sparse operations.

5. Fixed numerical settings

Setting	Supplied value
Tolerance	$1e-8$
Maximum iterations	5000
Repeats	3
Lambda sweep	$1e-6$, $1e-5$, $1e-4$, $1e-3$, $1e-2$, $1e-1$, 1.0

Setting	Supplied value
Precision	float64
Sparse storage	CSR
PCG preconditioner	Jacobi diagonal
PCG normal-equation product	Matrix-free

Changing these settings can be appropriate for a new study, but the result should then be identified as a modified experiment rather than an exact reproduction.

6. Memory and direct methods

Maragal_8 is a large rectangular rank-deficient problem. Direct methods that form or factorise large normal-equation systems are not the intended computational route. In practice, direct methods can fail because factorisation introduces fill-in and because rank deficiency complicates the factorisation. Increasing memory does not guarantee that a direct approach will become successful.

For the related overdetermined direct-solver cases in this dissertation repository, allocations below 128 GB were insufficient. That 128 GB figure is therefore a practical warning for those direct cases, not a universal guarantee or a requirement imposed on every Maragal_8 run. For Maragal_8 itself, direct solving is expected to be unsuitable in many configurations even when substantial memory is available.

Anyone attempting a direct comparison should check the current University memory allocation policy and establish the requirement for the actual solver and matrix. The supplied Maragal_8 benchmark does not require a direct solver.

7. CPU allocation

The number of CPUs is deliberately left user-dependent. The current CSF rules, partition and available resources determine what can be requested. The CPU allocation should be recorded with any reported timing results because runtime depends on the execution environment.

8. Slurm resources

Resource	Guidance
Wall time	24 hours was used in the dissertation workflow.
Memory	Check current CSF limits; do not assume a universal value.
CPUs	User-dependent; check current University limits.
Partition/account	Use current CSF instructions.
Array jobs	If used, keep task outputs separate and identifiable.

The 24-hour value describes the allocation used for the dissertation workflow. It does not replace the current University's resource policy.

9. Reproducibility restrictions

Keep fixed	Why
Matrix identity	Defines the numerical test problem.
RHS construction	Makes the benchmark deterministic.
Lambda sweep	Makes the solver comparison consistent.
Tolerance and iteration limit	Controls stopping behaviour.

Keep fixed	Why
PCG preconditioner	Changing it changes the PCG method.
Matrix-free formulation	Preserves the intended computational approach.
Precision and sparse format	Can affect memory, convergence and timing.

10. Timing and interpretation

Runtime is an empirical measurement, not a fixed property of the algorithm. CPU availability, system load, Python/SciPy versions and other environmental factors can change wall-clock time. When reporting results, record the resource request and software environment as well as the numerical settings.

The comparison should also consider iteration counts, residuals, normal-equation residuals, objective values and convergence status. A faster result is not automatically a better numerical result if convergence quality differs.

11. Generated data

The repository is intended to preserve the code needed to recreate the experiment. The Matrix Market archive, CSV results and PNG plots are generated or downloaded data and can be stored outside the source repository. Removing them from the repository does not remove the ability to reproduce the experiment, provided the scripts and instructions remain available.

12. Common problems

Problem	What to check
Matrix not found	Use <code>--download-data</code> or place <code>Maragal_8.mtx</code> below the specified data directory.
Import error	Activate a supported Python environment containing NumPy, pandas and SciPy.
Memory failure	Check the Slurm memory request and avoid explicit $A^T A$ formation.
Direct solver failure	Not unexpected for this scale/rank structure; the supplied experiment is iterative.
Timeout	Check the current wall-time limit and requested 24-hour allocation.
Plotting failure	Confirm that the benchmark CSVs exist before running the plotting script.
Different timing	Record CPU/memory allocation and software versions.

13. Final checklist

Before calling a run a reproduction, confirm that the correct `Maragal_8` matrix was used; `b` was generated as $A @ \text{ones}$; all default `lambda` values were tested; tolerance and maximum iterations were unchanged; PCG remained matrix-free with the Jacobi diagonal preconditioner; and the CSF resource allocation was recorded. Generated CSV and PNG files can then be retained as run outputs or regenerated when needed.